

Chapter 1: Your Kotlin Development Environment

By Irina Galata

Welcome to *Kotlin Apprentice*! In this first chapter, you're going to set up a development environment to let you program in the Kotlin language and work with the sample projects for each chapter in the book.

Then, you'll write your very first Kotlin code and see how to run the code on your machine.

The primary tool that you'll use in this book to create Kotlin projects is **IntelliJ IDEA** from **JetBrains**. JetBrains is also the company behind the Kotlin language itself, so Kotlin development is very tightly integrated into IntelliJ IDEA.

Note: Learn more about JetBrains at <http://www.jetbrains.com>.

IntelliJ IDEA is an **Integrated Development Environment**, or **IDE**, and is similar to other IDEs such as **Visual Studio** and **Xcode**. IntelliJ IDEA provides the foundation of many other IDEs from JetBrains, including **Android Studio** for Android app development, **PyCharm** for Python programming and **CLion** for C and C++ programming.

You use an IDE to write code in an editor, **compile** the code into a form that can be run on your computer, see output from your program, fix issues in your code and much more! You'll just scratch the surface of the power of IntelliJ IDEA in this chapter, but you'll be setup to work with the code examples throughout the rest of the book.

Getting started with IntelliJ IDEA

You can download IntelliJ IDEA from the JetBrains website at <https://www.jetbrains.com/idea/>. There are both **Community** and **Ultimate** editions of the IDE; you'll just need the Community edition to work with the code in this book. The Community edition is a free download.

Go ahead and download IntelliJ IDEA 2020.3 or later on your platform of choice. There are versions for macOS, Windows and Linux. Follow the installation instructions on the JetBrains site to install IntelliJ IDEA on your machine. Most of the screenshots in this book will be from the macOS version, but the Windows and Linux versions are similar.

Before you first run IntelliJ IDEA, you'll also want to install a **Java Development Kit**, or **JDK**, which will easily let you run Kotlin code on your machine.

Java and the JDK

Kotlin allows you to program on a number of different platforms. The two most prominent platforms are the **Java Virtual Machine**, or **JVM**, and **Android**. See Chapter 26: “Kotlin Multplatform” for more information on all the different platforms that Kotlin runs on.

In many ways, Kotlin was initially created as a modern replacement for the **Java** programming language. Java was created in the 1990's as an early attempt at a **cross-platform** application language, promising a “Write Once, Run Everywhere” approach to software development.

Instead of compiling to native machine code on each platform, Java programs are compiled into a format called **bytecode**. The bytecode runs inside an application on the Java Virtual Machine. The JVM can be thought of as a layer above your actual machine. By running as bytecode on a virtual machine, you are able to share Java code and applications across many types of computer systems.

One of the goals of the Kotlin programming languages is to be 100% **interoperable** with the Java language. This includes Kotlin code being converted to Java-compatible bytecode by the Kotlin compiler, so that the Kotlin code can be run on the JVM.

Most of the code and projects in this book are meant to be run as Kotlin projects on the JVM. In order to do so, you must install the JDK alongside IntelliJ IDEA. The easiest way to get a JDK for your platform is to visit the Oracle website at:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

You'll want to download and install the latest version of the JDK — at least version 8. The Java software tools go by the name “Java SE,” which includes the JDK and also the **Java Runtime Environment**, or **JRE**.

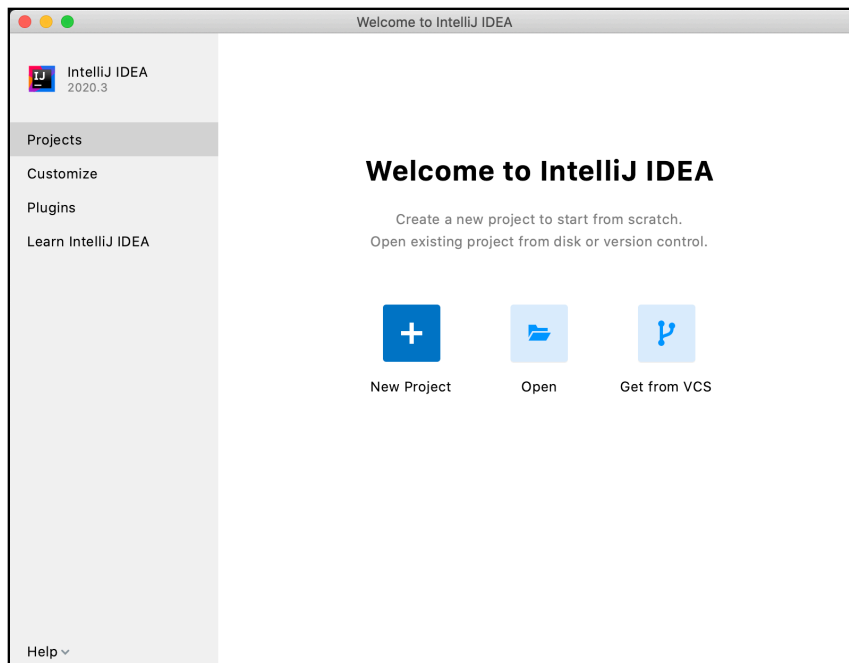
Note: Be sure to download and install the JDK and not just the JRE, since the JRE only lets you run Java applications and does not include the tools to build new ones.

Running IntelliJ IDEA

Once you've installed IntelliJ IDEA and the JDK, follow the normal process of starting the IntelliJ IDEA application on your platform.

If you've installed previous versions of IntelliJ IDEA on the same machine, the installer will likely prompt you to import settings from a previous version. If you've not installed previous versions on the same machine, you'll be prompted to choose a color theme and choose plugins to install into the IDE. You can just choose the default settings and proceed.

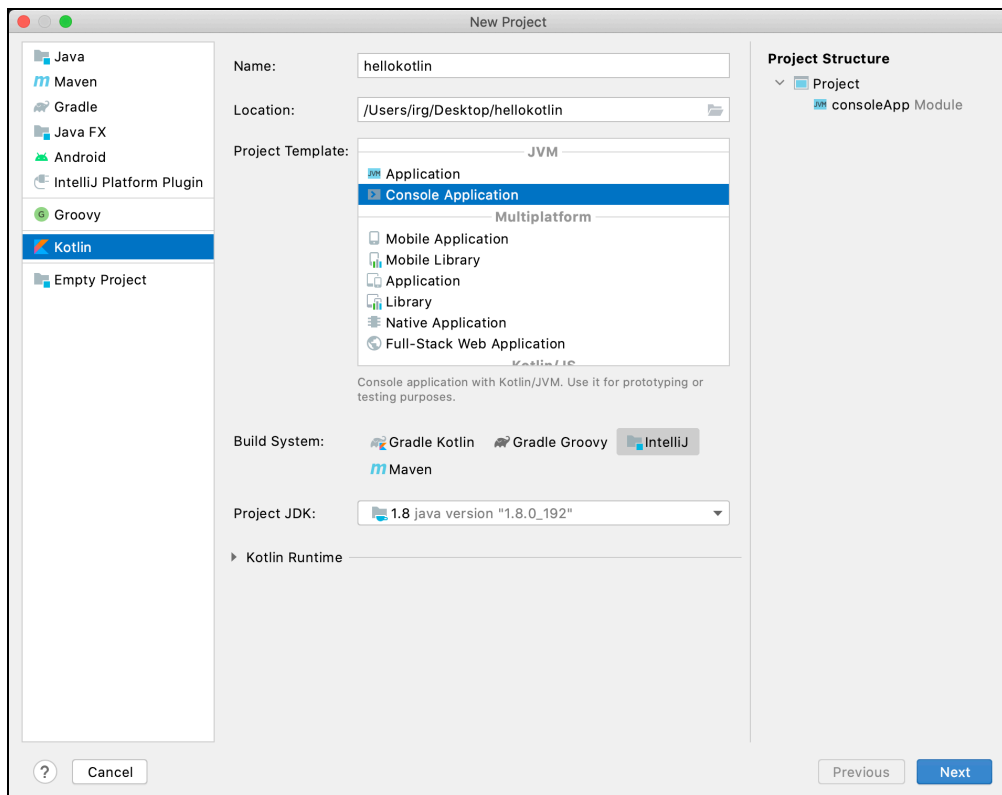
You'll then arrive at the **Welcome to IntelliJ IDEA** window.



From the welcome window, you can create new projects, import or open existing projects, check out code from a version control system such as **Git**, run configuration tools and get help on the IDE.

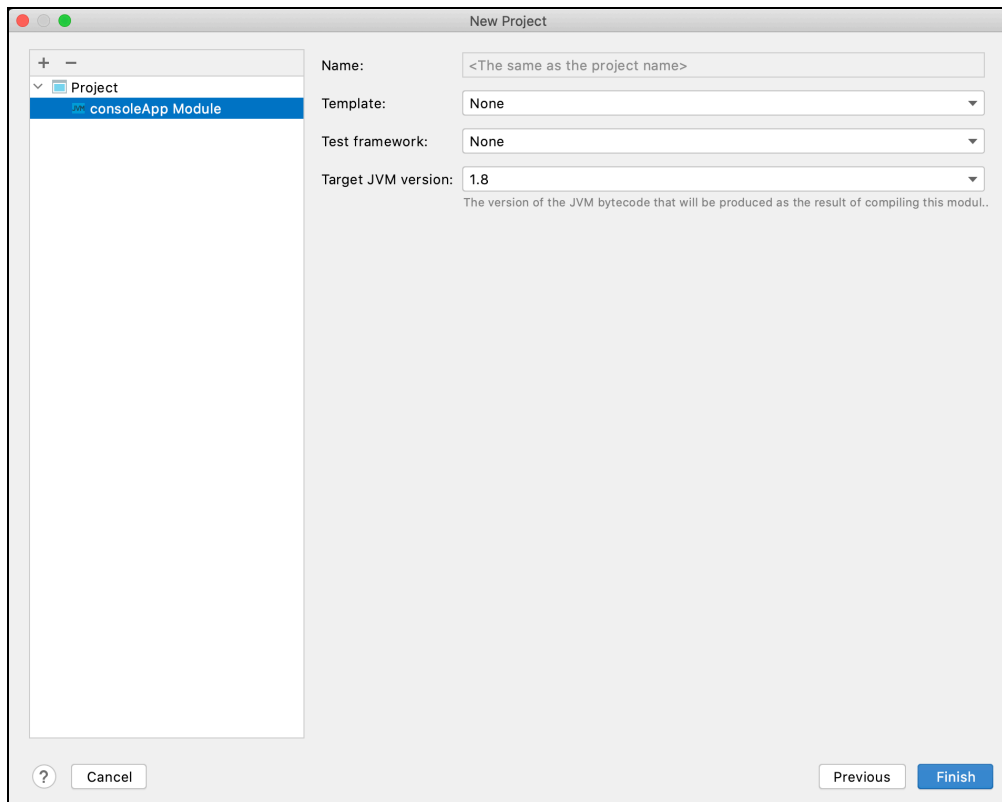
Your first project

Go ahead and choose **New Project** on the welcome screen. You'll see the first of two project configuration screens.



Choose **Kotlin** in the list of options on the left and **Console Application** as the project template. Type in **hellokotlin** as the project name, and choose a project location or accept the default, pick **IntelliJ** as a build system. You also see the **Project JDK**, which should be the JDK version that you installed earlier — or a different JDK version if you have more than one installed on your machine. When you are ready click **Next**.

You'll see the following:

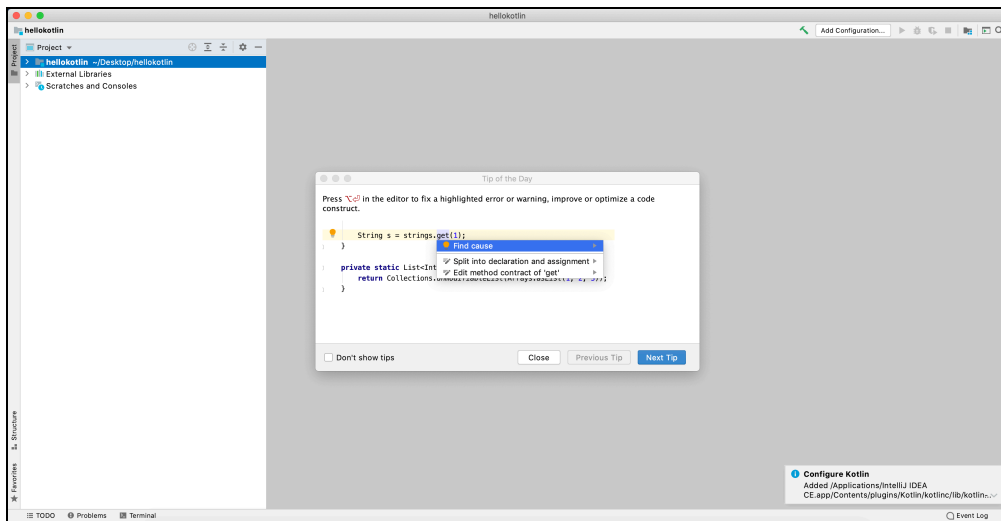


Pick **None** for both template and the test framework, later on, you will use different options for a faster setup of your project. Now click **Finish**.

At this point, IntelliJ IDEA will create and configure the project for you.

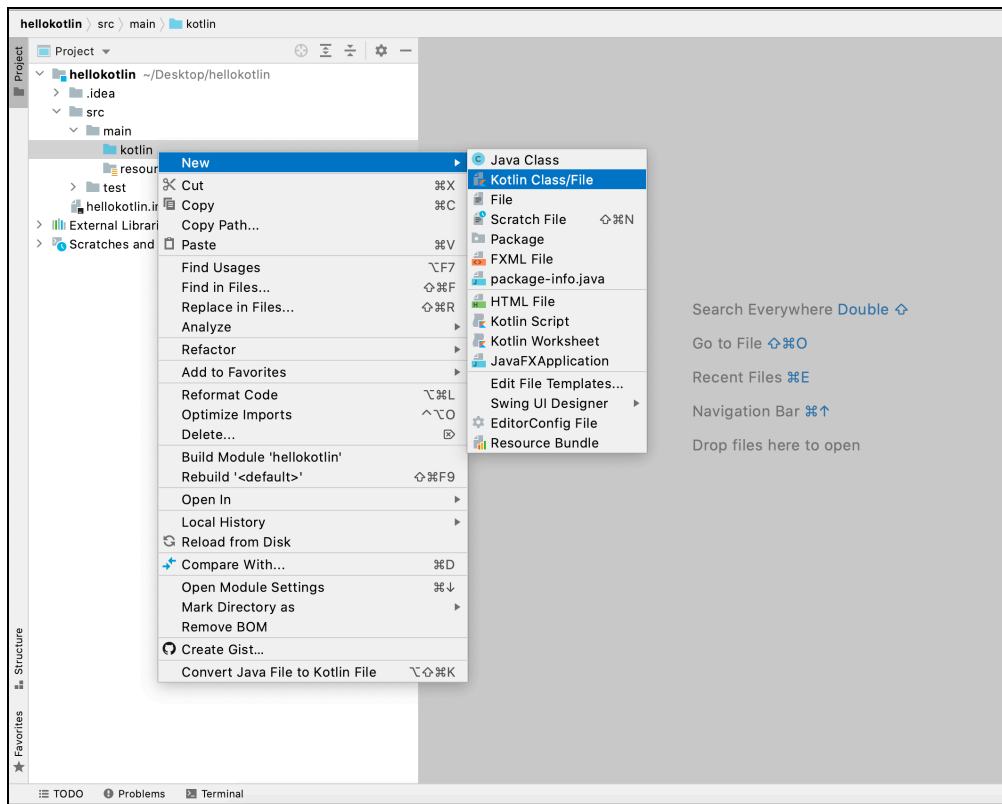
When it's finished, you'll arrive at a **Tip of the Day** window, which gives you helpful IntelliJ IDEA tips each time you open the application.

You'll see the following:

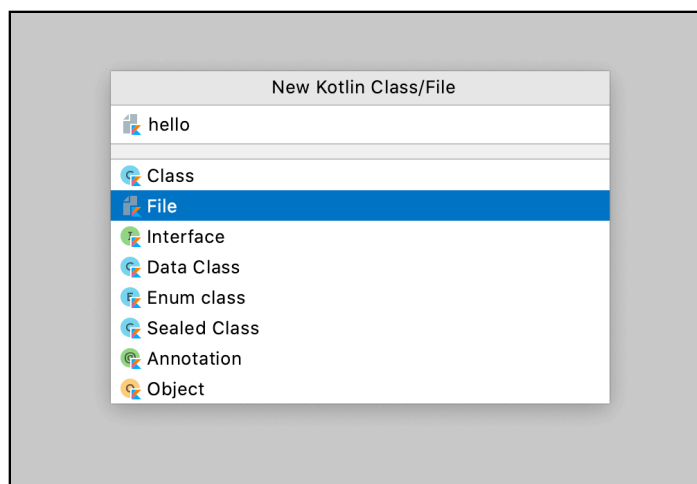


Close the tip window, and check out the **Project** panel on the left of the main IntelliJ IDEA window. The Project panel is where you manage all the files associated with the project, such as your Kotlin source code files, which end with a **.kt** file extension.

Click the arrow next to **hellokotlin** to reveal its contents, and you'll see a **src** folder for the project and its subfolders. Expand the **src** and **main** folders. Right-click on the **kotlin** folder and choose **New ▶ Kotlin File/Class**.

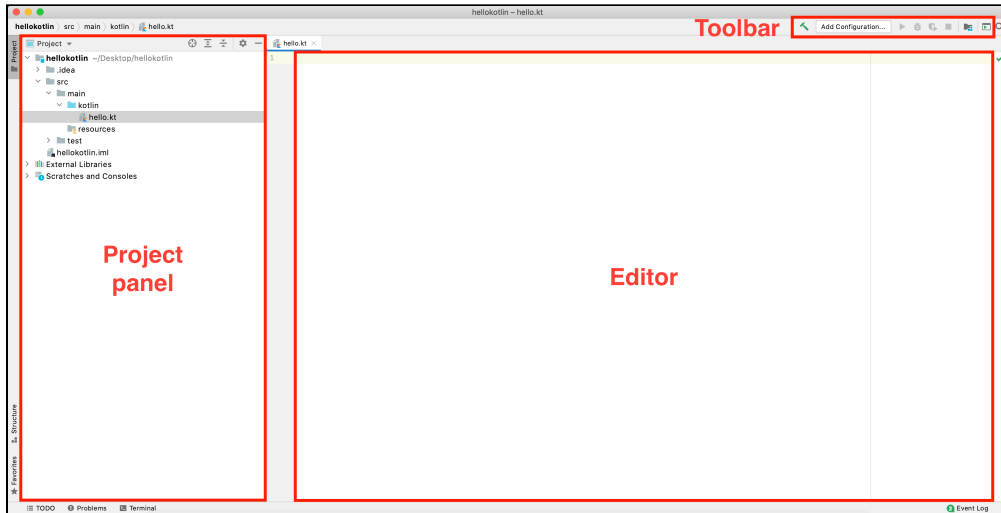


The **New Kotlin File/Class** dialog will open. Enter the name **hello** and double-click **File**.



The file **hello.kt** will then open in the IntelliJ IDEA editor.

The basic layout of the IntelliJ IDEA window contains the Project panel on the left, the **Editor** panel in the middle, and a **Toolbar** in the upper right that you can use to run your code.



Now that your project is setup and you know the main parts of the IntelliJ IDEA window, it's time to run some Kotlin code!

Hello, Kotlin!

For this first chapter, you'll type some Kotlin code into an editor and run it, without necessarily understanding all the parts of the code. You'll learn more about the code you're typing as you proceed in the book. If you have experience with other programming languages, such as Java, Swift or Python, then the code might look familiar.

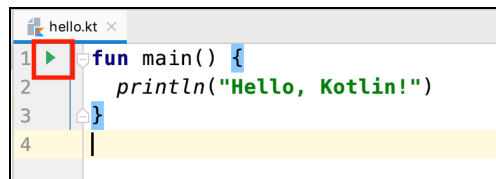
In the Editor panel for the file **hello.kt**, type in the following code exactly as written:

```
fun main() {  
    println("Hello, Kotlin!")  
}
```

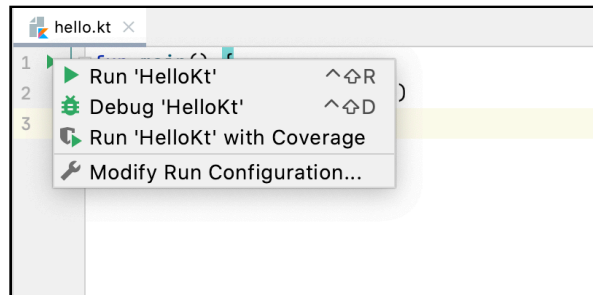

You’ve written a single Kotlin **function** named `main()`, and added a single line of code to the function inside the braces, which then calls another function named `println()`. You’re telling Kotlin to print the text “Hello, Kotlin!” to the screen. You’ll learn much more about Kotlin functions later in the book.

There are a few different ways you can run this code, including using the IntelliJ IDEA menu, using the toolbar and using certain keystrokes.

The easiest way to run the code is to click the little green Run/Play button to the left of the `main()` function in the Editor panel. It may take a moment to show up after you add the code.



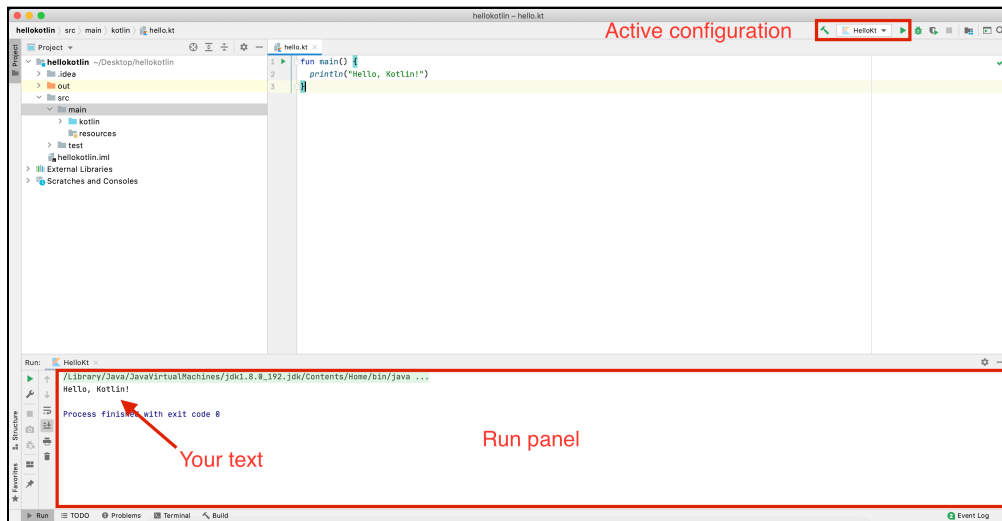
Go ahead and click the green Run button and a menu will open. Choose **Run 'HelloKt'** from the menu.



When you do, the Kotlin compiler will parse your code and convert the code to bytecode, and it will run it on your local JVM.

A panel will then open at the bottom of the IntelliJ IDEA window named the **Run** panel, sometimes also called the **console**.

You'll see the program output in the Run panel — in this case, the text that you wanted to show on the screen.



After this first run of the code in your project, you now have an active project **configuration** in the toolbar, and you can run the code by tapping the green Run button in the toolbar.

Nice! You've created your first Kotlin project and run your first Kotlin program!

Book sample projects

The sample code for each chapter of the book, except for the last three chapters, Chapter 24: “Scripting with Kotlin”, Chapter 25: “Kotlin/Native”, and Chapter 26: “Kotlin Multiplatform”, falls into two categories: **Kotlin JVM** projects and **Gradle** projects. The vast majority are the simpler Kotlin JVM projects. The two types of projects are opened in a similar way, as described in this section.

Kotlin JVM projects

Kotlin JVM projects are just like the one you created in the last section of this chapter. For most of the chapters in the book, the projects look just like that project, with a single `main()` function in which you can put the Kotlin code you need to run. In certain cases, you may need to add code outside the `main()` function in the editor, and that will be pointed out when needed.

As you work through these chapters, you can either open the **starter** project for the chapter, which will have an empty `main()` function to which you can add code, or you can create a new project to work with, as you did in the last section.

In either case, you just enter code as you work your way through the chapter. You press the Run button in IntelliJ IDEA to run the code in the project at any point.

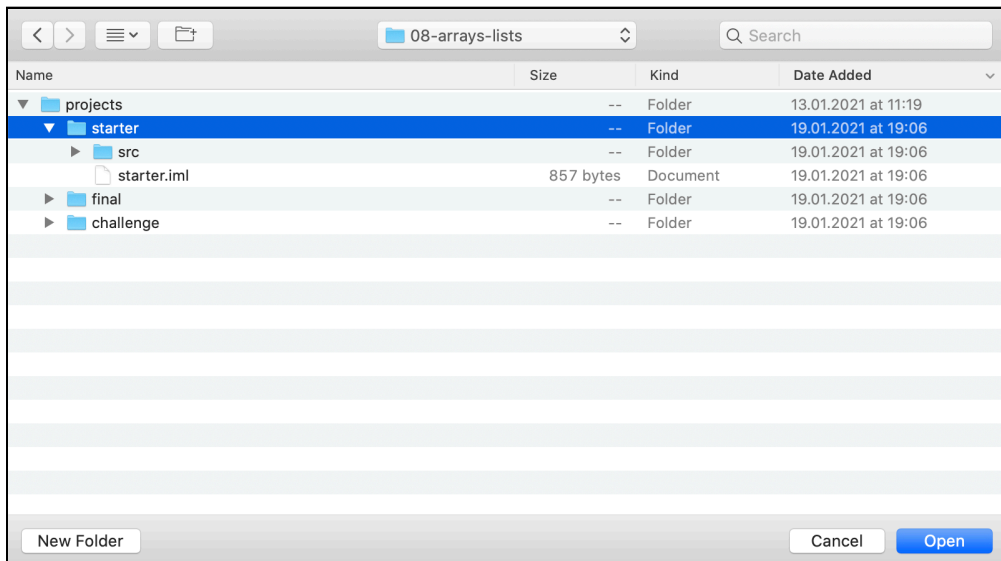
If you choose to create your own projects, you can always open the chapter sample code in a text editor if you want to see the code yourself, in order to address any issues you have when entering the code. Just open the files that end with the extension **.kt** in a text editor like **Notepad** on Windows or **TextEdit** on macOS.

Try to avoid copy-and-pasting the code from a text editor into IntelliJ IDEA though, since typing in the code yourself helps to solidify your knowledge.

If you choose to open the starter project instead of making your own, do so by clicking **Open** from the **Welcome to IntelliJ IDEA** window, or selecting **File ▶ Open** from the IntelliJ IDEA menu.

You then just need to choose the root folder for the project, e.g., the folder named **starter** for opening the starter project, and click **Open**.

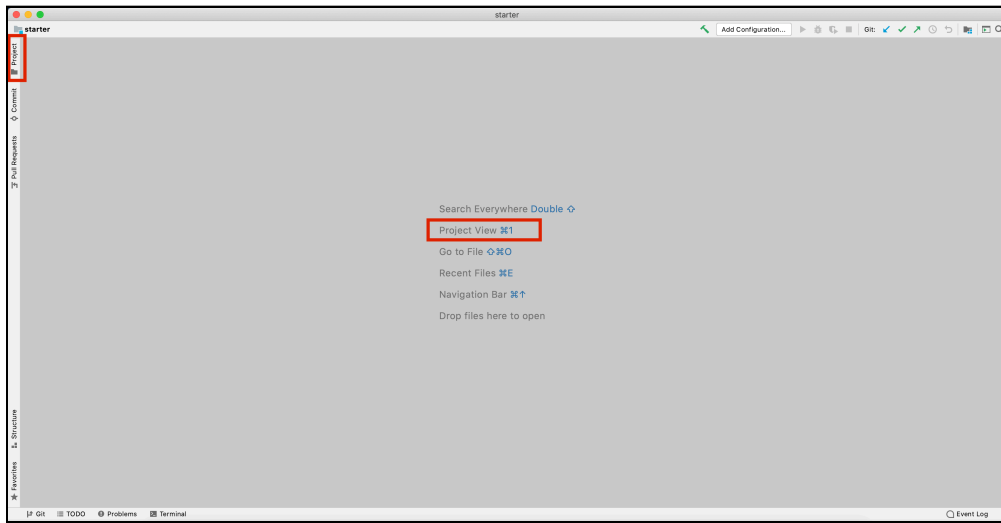
You'll see the following:



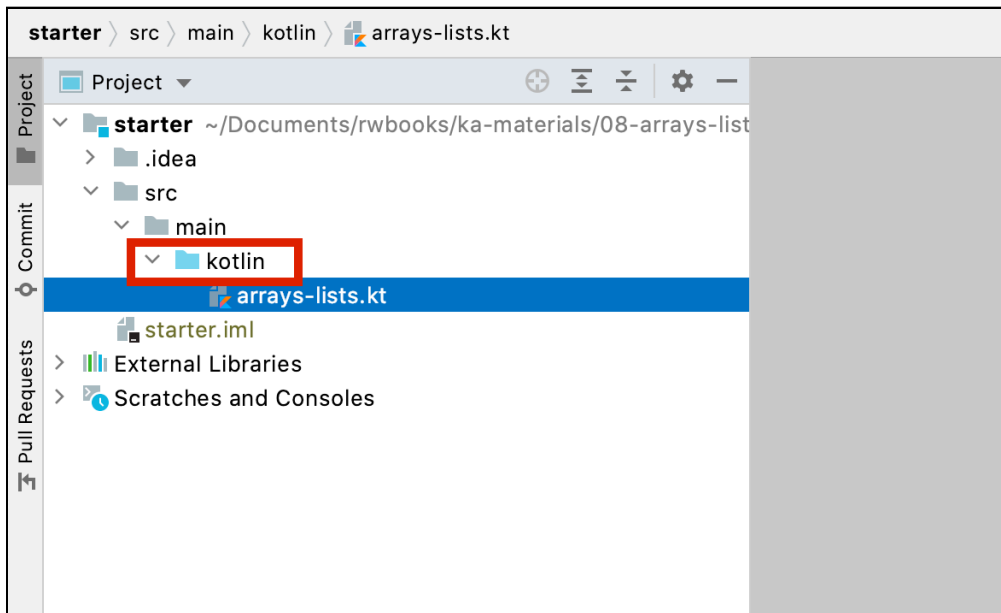
IntelliJ IDEA will then open the project, and you can start entering code as if you had created the project yourself.

When the project opens, you may need to select **View ▶ Tool Windows ▶ Project** to open the Project panel.

You can also click the Project tool button in the upper left of the IntelliJ IDEA window, or press **command-1** on Mac or **Alt-1** on PC to show the Project panel:



Once the Project panel is open, expand the root project and open up the **kotlin** folder to find the Kotlin source code files for the project:



Gradle projects

For a couple of the chapters, the code projects use Gradle, in order to allow pulling an external dependency into the project.

Gradle is a **build system** and **dependency management** tool that is popular within the Java ecosystem. It's an extremely powerful and versatile build tool, and its power goes well beyond our purposes in the book.

Gradle is used as the build system for Android apps built using Android Studio, which, as was mentioned earlier, is based on IntelliJ IDEA.

To open the Gradle projects in Chapter 23, you use the exact same steps as for the Kotlin JVM projects. You choose **File ▶ Open** and then navigate to and select the root folder of the project.

IntelliJ IDEA will detect that the project is Gradle-based and then open and configure the project accordingly.

Final projects and challenges

In addition to a starter project, each chapter also has a folder for the **final** project and a folder with solutions to the **challenges** you find at the end of each chapter. You can open the final and challenge projects in the same manner as described above for the starter projects. The challenge projects also contain solutions to **Mini-Exercises** that you come across in the chapters.

Challenges

Challenges are a key part of working through the *Kotlin Apprentice*. Each chapter contains some challenges at the end of the chapter, and most chapters contain mini-exercises in the text of the chapter. Solving the challenges and mini-exercises will enhance and enforce the knowledge you've learned in each chapter.

As your challenge for this first chapter, make sure you've both run through the process of creating a new Kotlin JVM project, as well as opened an existing project from the final project folder for Chapter 1. Make sure to run the code in both cases. Doing so will ensure that you can run the sample code in the rest of the book.

Key points

- **IntelliJ IDEA** is an **Integrated Development Environment** from JetBrains, the creators of the Kotlin language, in which you can write and run Kotlin code.
- IntelliJ IDEA **Community edition** is a free version to use for the projects in the book.
- Kotlin code runs on many platforms, and one of the most prominent is the **Java Virtual Machine**, which will be used for most of the book.
- To build Kotlin projects with IntelliJ IDEA, you need to install the **Java Development Kit**, version 8 or above.
- The IntelliJ IDEA app window consists of a number of panels, the most relevant of which are the **Project** panel, the **Editor** panel, and the **Run** panel.
- The book starter, final and challenge projects can be opened by choosing **File ▶ Open** from the IntelliJ IDEA menu and selecting the root folder of the corresponding project.

Where to go from here?

There's a lot more to explore in IntelliJ IDEA, including debugging, refactoring, code profiling and version control system integration. You can find out more about these features of IntelliJ IDEA in the official JetBrains documentation.

Now that you're setup with a development environment, in Chapter 2, "Expression, Variables & Constants," you'll first get a primer on how computers work, and then you'll start writing some real Kotlin code!